Project_PoC

October 30, 2026

[1]:
```
NAMES = ['Angela DiAndreth', 'Sergey Dutt', 'Edward Lombo-Becerra']
```

[2]:
```
import ipympl
from IPython import get_ipython
from IPython.display import display
import ipywidgets as widgets
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from scipy.integrate import solve_ivp
from scipy.optimize import curve_fit

from fix_data import get_random_subset, load_and_center_data



np.random.seed(42)  # for reproducibility
```

1 Project Title: Create a Pendulum Model with Less Sensitive Parameters

2 Statement of whether the proof of concept matches the proposal (yes/no):

Our proof of concept mostly matches our original proposal, focusing specifically on modeling pendulum trajectories and applying the Fisher Information Matrix (FIM) to analyze parameter correlation and model sensitivity. The rationale for selecting the pendulum system and changes in our intended implementation of FIM are further detailed in the "Explanation of Major Changes" section.

3 Discussion of Critical Challenge:

The critical challenge of this project is decoupling intrinsic physical parameters (length L, mass M, and radius r) from environmental damping parameters (air-resistance coefficient γ) without sacrificing trajectory fit. In standard pendulum models, L and γ are highly correlated. During

model calibration, this coupling forces the parameter optimizer to inflate L beyond its true measured value to compensate for errors in γ, producing non-physical parameter estimates.

This correlation is particularly problematic for engineering design. Typically, an engineer designs a system around a target environmental damping requirement (γ) and relies on simulations to determine the necessary intrinsic parameters (L,M,r). If the simulation model exhibits strong parameter coupling, the optimizer will yield biased physical values that fail when built in the real world.

Our project will try to fix this issue by developing a model formulation that structurally separates these two parameter classes. To demonstrate this separation, we perform parameter estimation across both the unknown environmental parameter (γ) and a baseline physical parameter (L). We then evaluate the model across three metrics: trajectory fitting accuracy, the degree of correlation between intrinsic and environmental parameters, and the error between the optimized intrinsic parameter value and its known physical measurement.

4 Mathematical basis:

A standard model for a simple pendulum subject to linear air resistance is given by the second-order non-linear differential equation:

$$\frac{d^2\theta}{dt^2} + \gamma\frac{d\theta}{dt} + \frac{g}{L}\sin\theta = 0 \tag{1}$$

$\theta(t)$: Angular displacement from the vertical equilibrium position (in radians) as a function of time t.

$\frac{d\theta}{dt}$: Angular velocity (ω), representing how fast the angle changes over time.

$\frac{d^2\theta}{dt^2}$: Angular acceleration (α), representing the rate of change of angular velocity.

γ: Damping parameter per unit mass ($\gamma = \frac{c}{m}$ where c is the drag coefficient and m is the pendulum mass)

g: Acceleration due to gravity (≈ 9.81 m/s^2).

L: Effective length of the pendulum from the pivot point to the center of mass.

Reference: Taylor, John R. Classical Mechanics. University Science Books, 2005. Chapter 5: Oscillations (Section 5.4: Damped Oscillations).

To solve this second-order differential equation numerically using scipy.integrate.solve_ivp, it must be turned into a system of two coupled first-order ordinary differential equations:

$$\frac{d\theta}{dt} = \omega$$
$$\frac{d\omega}{dt} = -\gamma\omega - \frac{g}{L}\sin\theta$$

5 Simplifications for Proof of Concept:

Our proof of concept demonstrates the coupling between physical parameter L and damping parameter γ under the standard model from the previous section. This analysis does not require the

full dataset; a subset with a few oscillations suffices to show the model's mechanics when optimizing γ alone, and when optimizing both γ and L. This relies on the assumption that the range of motion shrinks with each oscillation throughout the entire dataset, which holds true from start to finish.

6 Solution to Reduced Problem:

Pendulum.xlsx contains experimental pendulum motion data collected from an undergraduate physics project at Luther College. The angular position relative to the equilibrium position was recorded at an interval of 0.1 seconds over approximately one minute. The workbook contains multiple sheets corresponding to different wire thicknesses holding the pendulum mass.

This notebook uses the "Clear" sheet (representing the thickest wire) due to its superior measurement accuracy across the dataset. Because the sampling rate is uniform in time, data points are not evenly spaced in angle. The slow velocity near turning points yields a higher density of measurements near local maxima and minima than near the equilibrium crossing points.

This notebook demonstrates that pendulum motion models with air resistance, γ, (optimized against the data as the sole free parameter) struggle to accurately capture the true trajectory of the pendulum, even over a small window of just a few oscillations. However, if you have both air resistance (γ) and length of pendulum (L) as free variables, it provides a nearly 1-to-1 fit.

[3]:
```
#Note: to run this .ipynb, you have to drag Pendulum.xlsx into the files tab
df = load_and_center_data('Pendulum.xlsx')
subset_df = get_random_subset(df, 15)

# Subtract the first time value of the subset (reframes to t_0 = 0)
subset_df['Time'] = subset_df['Time'] - subset_df['Time'].iloc[0]

# find first max and min (they will be the global max and min)
first_max = subset_df['Angle'].max()
first_min = subset_df['Angle'].min()
```

Since the mathematical models assume that the equillibrium is at $\theta = 0$ and the recorded data in `Pendulum.xlsx` off-centered, `load_and_center_data` realigns the equilibrium point across the file.

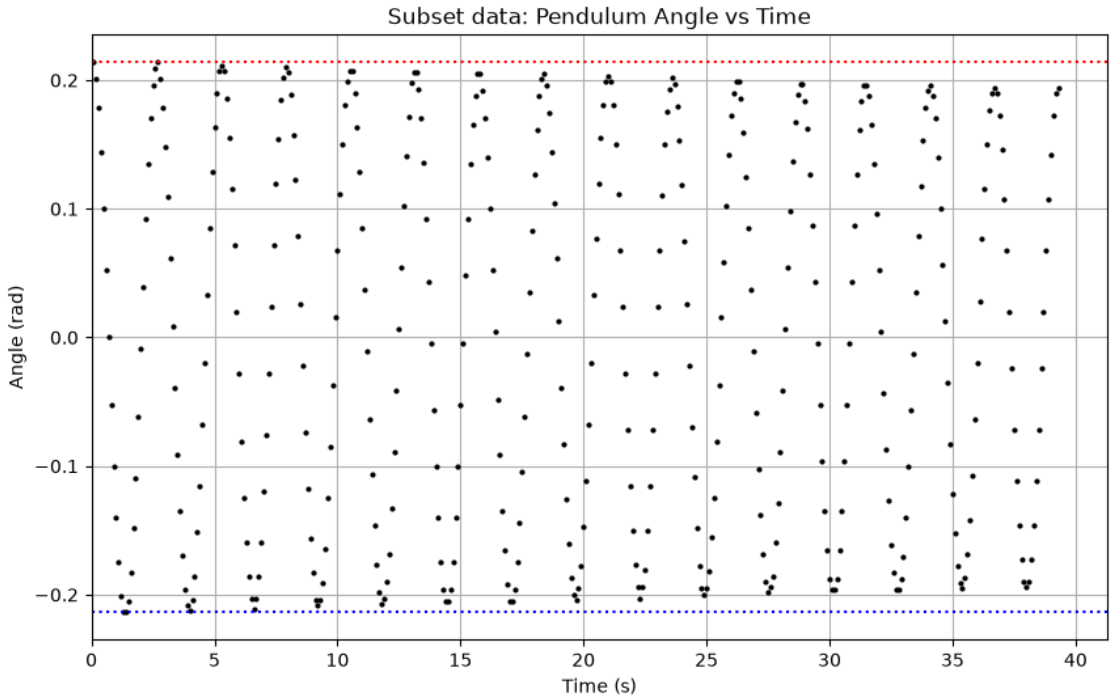
For the purposes of the proof of concept, we will use subsets of the full dataset. Since the spreadsheet is very large (4936 datapoints), `get_random_subset` selects a random group of points that starts at a local max and meets a specified oscillation requirement (~395 datapoints on average).

[4]:
```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(subset_df['Time'], subset_df['Angle'], 'ko', markersize = 2)

ax.set_xlim(left=0)
ax.hlines([first_max, first_min], *ax.get_xlim(), colors=['r', 'b'],␣
 ↪linestyles=':')

ax.set_xlabel('Time (s)')
ax.set_ylabel('Angle (rad)')
ax.set_title('Subset data: Pendulum Angle vs Time')
```

```
ax.grid();
```



Using the horizontal lines as guides, you can see that the maximum angular displacement is slowly decreasing, i.e. decaying after each oscilation.

We now implement the pendulum model from the FODE (2) in a function. As well as the solver, using `solve_ivp` to obtain a solution for $\theta(t)$

[5]:
```
def pendulum_model(t, y, gamma, L = 1.615, g = 9.81):
    """
    Parameters:
    t: current time point
    y: state vector [theta, omega]
    - theta is angular displacement
    - omega is angular velocity
    gamma: damping parameter
    L: length of the pendulum (1.615)
    g: acceleration due to gravity (9.81)
    """
    theta, omega = y
    dtheta_dt = omega
    domega_dt = -gamma * omega - (g / L) * np.sin(theta)

    return [dtheta_dt, domega_dt]
```

[6]:
```python
def solve_pendulum_ivp(t_eval, *params):
    theta_0 = subset_df['Angle'].iloc[0]
    omega_0 = 0 # no initial velocity
    y0 = [theta_0, omega_0]

    sol = solve_ivp(
        fun = pendulum_model,
        t_span = (t_eval[0], t_eval[-1]),
        y0 = y0,
        t_eval = t_eval,
        args = params, # args must be a tuple
        method = 'RK45',
    )

    return sol.y[0] # return theta values
```

[7]:
```python
# Plotting function for reusable plotting of data and model fit
def plot_model(time_data, angle_data, model_fun, fit_params, description = ''):
    """
    Generates plot of with model line and data points
    Works with 1 or multiple free parameters
    """
    plt.close('all') # close any open figures
    t_model = np.linspace(time_data.iloc[0], time_data.iloc[-1], 500)

    if isinstance(fit_params, (list, tuple, np.ndarray)):
        params = tuple(fit_params)
    else:
        params = (fit_params,)

    y_model = model_fun(t_model, *params)

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(
        time_data,
        angle_data,
        'ko',
        markersize = 3,
        label = 'Data',
    )

    # plot modeled trajectory
    ax.plot(t_model, y_model, color = 'crimson', linewidth = 2, label = 'Model')

    ax.set_xlabel('Time (s)')
    ax.set_ylabel('Angle (rad)')
    ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
```

```
    fig.suptitle(f'Pendulum Angle vs Time for {description}')
    plt.show()
```

First we solve the problem with air resistance (γ) as the sole free parameter.

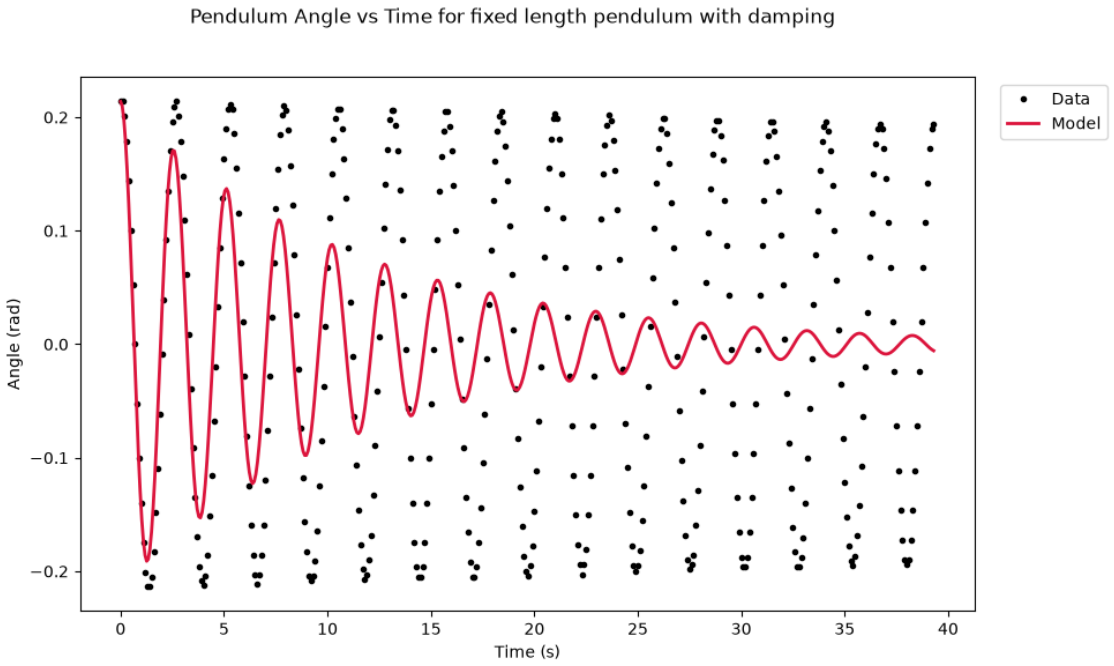
[8]:
```
# popt = optimal parameter value
# pcov = parameter covariance matrix
popt_1p, pcov_1p = curve_fit(
    solve_pendulum_ivp,
    subset_df['Time'].values,
    subset_df['Angle'].values,
    p0 = [0.05], # initial guess (array)
    bounds = (0, 1.0), # bounds (tuple)
)

# gamme _err = std deviation of gamma
gamma_fit = popt_1p[0]
gamma_err = np.sqrt(pcov_1p[0][0])

print(f'Optimized Damping Coefficent (gamma): {gamma_fit:.5f} ± {gamma_err:.5f}␣
  ↪s^-1')

plot_model(
    time_data = subset_df['Time'],
    angle_data = subset_df['Angle'],
    model_fun = solve_pendulum_ivp,
    fit_params = gamma_fit,
    description = 'fixed length pendulum with damping'
)
```

```
Optimized Damping Coefficent (gamma): 0.17315 ± 0.02714 s^-1
```



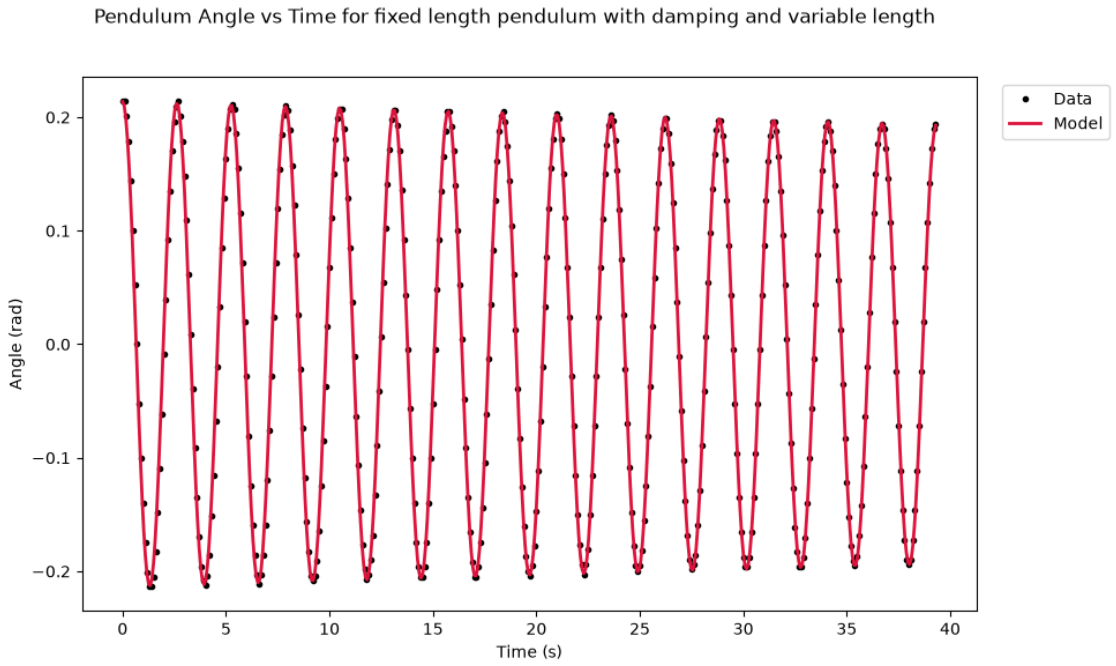
Now, we solve it with both air resistance (γ) and length of pendulum (L) as free parameters.

[9]:
```
popt_2p, pcov_2p = curve_fit(
    solve_pendulum_ivp,
    subset_df['Time'].values,
    subset_df['Angle'].values,
    p0=[0.05, 1.615],  # initial guesses: gamma=0.05, L=1.615m
    bounds=([0.0, 1.0], [1.0, 2.0]),
)

gamma_fit, L_fit = popt_2p
gamma_err, L_err = np.sqrt(np.diag(pcov_2p))
print(f'Optimized Damping Coefficents -> gamma: {gamma_fit:.5f} ± {gamma_err:.
 ↪5f} s^-1')
print(f'                              -> L:     {L_fit:.5f} ± {L_err:.4f} m')

plot_model(
    time_data = subset_df['Time'],
    angle_data = subset_df['Angle'],
    model_fun = solve_pendulum_ivp,
    fit_params = (gamma_fit, L_fit),
    description = 'fixed length pendulum with damping and variable length'
)
```

```
Optimized Damping Coefficents -> gamma: 0.00493 ± 0.00029 s^-1
                              -> L:     1.70107 ± 0.0002 m
```



[10]:
```
#We created an interactive pendulum graph, to show how the curve-fitting␣
 ↪changes as you change L and
#It helps to visualise what curve_fit and solve_ivp are doing here, and shows␣
 ↪the intermediary values as L/ converge to their optimums.
#Unfortunately this will not run with a pdf, but if you open the .ipynb file␣
 ↪you can slide them!
#Google Gemini aided with trouble-shooting (helped with figuring out which␣
 ↪package to use, and helped with the making gamma slider more accurate)
def interactive_pendulum(time_data, angle_data):
    get_ipython().run_line_magic('matplotlib', 'widget')
    #Time array
    t_model = np.linspace(time_data.iloc[0], time_data.iloc[-1], 500)

    #Define a function that just does the plotting for a given L
    fig, ax = plt.subplots(figsize=(8, 6))
    #Plot raw data
    ax.plot(
        time_data, angle_data,
        color='black', marker='o', linestyle='none',
        markersize=3, label='Data'
    )
    model_line, = ax.plot(
        t_model, np.full_like(t_model, np.nan),
        color='crimson', linewidth=2, label='Model'
    )
```

```
    ax.set_xlabel('Time (s)')
    ax.set_ylabel('Angle (rad)')
    ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0.0)
    fig.tight_layout()

    def plot_update(gamma_val,L_val):
        #Plot raw data
        #Calculate new model data using the updated values
        y_model = solve_pendulum_ivp(t_model, gamma_val, L_val)

        #Plot the model line based on the changes.
        model_line.set_ydata(y_model)
        ax.relim()
        ax.autoscale_view()
        fig.canvas.draw_idle()
    #Gamma-slider
    G_slider = widgets.FloatSlider(
        value=0.005, #This is the (rounded) value of gamma that was found to be␣
↪optimal by curve_fit
        min=0.0,
        max=0.1,
        step=0.001,
        readout_format='.3f', #So the gamma slider is accurate to the 0.00X.
        description='Gamma ( ):',
        continuous_update=False  #Lag prevention
    )
    #L-slider values
    L_slider = widgets.FloatSlider(
        value=1.70, #This is the (rounded) value of L that was found to be␣
↪optimal by curve_fit
        min=1.65,
        max=1.75,
        step=0.01,
        description='Length (L):',
        continuous_update=False  #Lag prevention
    )

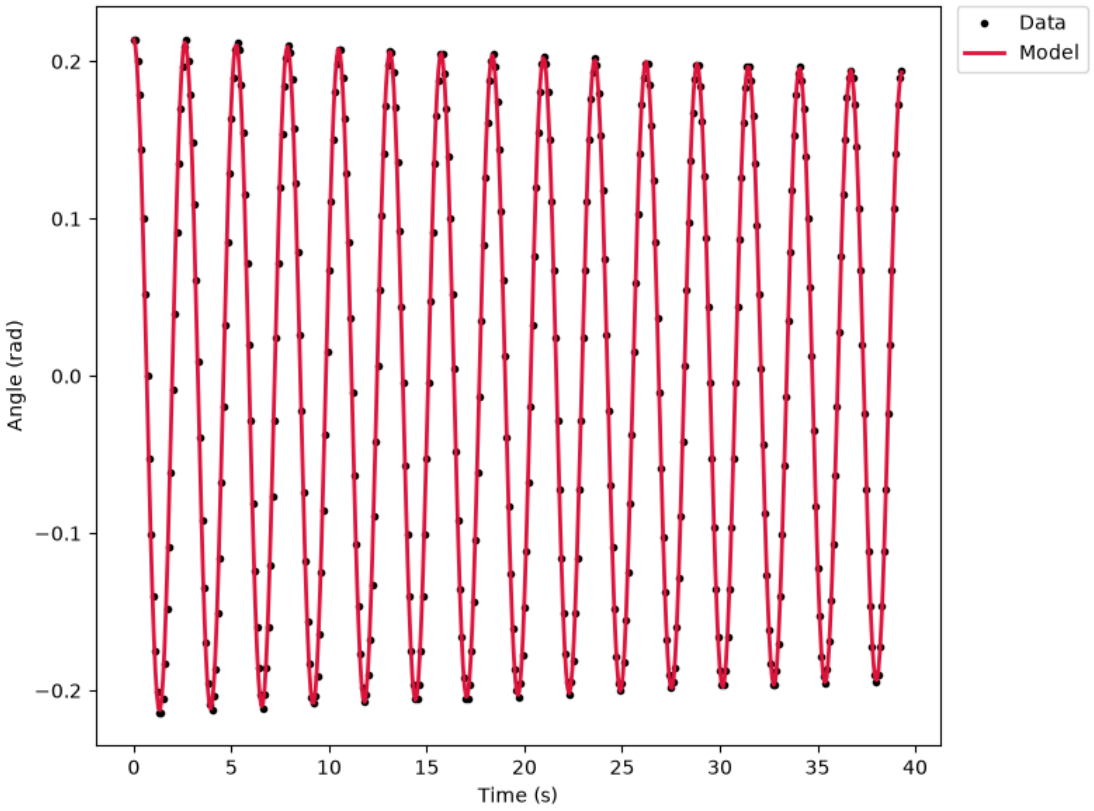
    #Link the sliders to the plotting function
    def update_from_sliders(change):
        plot_update(G_slider.value, L_slider.value)

    G_slider.observe(update_from_sliders, names='value')
    L_slider.observe(update_from_sliders, names='value')
    plot_update(G_slider.value, L_slider.value)
    display(widgets.VBox([G_slider, L_slider]))
    display(fig.canvas)
    get_ipython().run_line_magic('matplotlib', 'inline')
```

[11]:
```
#Run curve fit and solve_ivp functions
popt_2p, pcov_2p = curve_fit(
    solve_pendulum_ivp,
    subset_df['Time'].values,
    subset_df['Angle'].values,
    p0=[0.05, 1.615],
    bounds=([0.0, 1.0], [1.0, 2.0]),
)
gamma_fit2, L_fit = popt_2p
gamma_err2, L_err = np.sqrt(np.diag(pcov_2p))

#Plot the interactive plot!
interactive_pendulum(
    time_data=subset_df['Time'],
    angle_data=subset_df['Angle'],
)
```

```
VBox(children=(FloatSlider(value=0.005, continuous_update=False,␣
↪description='Gamma ( ):', max=0.1, readout_fo…
```



Uppon closer inspection with this widget tools, it can be shown that many (γ, L) pairs produce

nearly indistinguishable predictions. Meaning the different damping and length values can reproduce the same observed motion. The data fit looks good, but the individual parameter values are not trustworthy. FIM would quantify that the data cannot independently distinguish those parameters.

For each fixed L, we fit only γ, record the optimized $\gamma(L)$, and plot the resulting curve (in the 3d plot, the error is shown as well).

[12]:
```
# correlation between L and gamma
L_values = np.linspace(1.65, L_fit + 0.025, 30)

gamma_values = []
gamma_errors = []

for L_val in L_values:
    popt, pcov = curve_fit(
        lambda t,g: solve_pendulum_ivp(t, g, L_val),
        subset_df['Time'].values,
        subset_df['Angle'].values,
        p0 = [0.05],
        bounds = [0, 1],
    )

    g_fit = popt[0]
    g_err = np.sqrt(pcov[0][0])

    gamma_values.append(g_fit)
    gamma_errors.append(g_err)
```

[13]:
```
from mpl_toolkits.mplot3d import Axes3D
plt.close('all') # close any open figures
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')

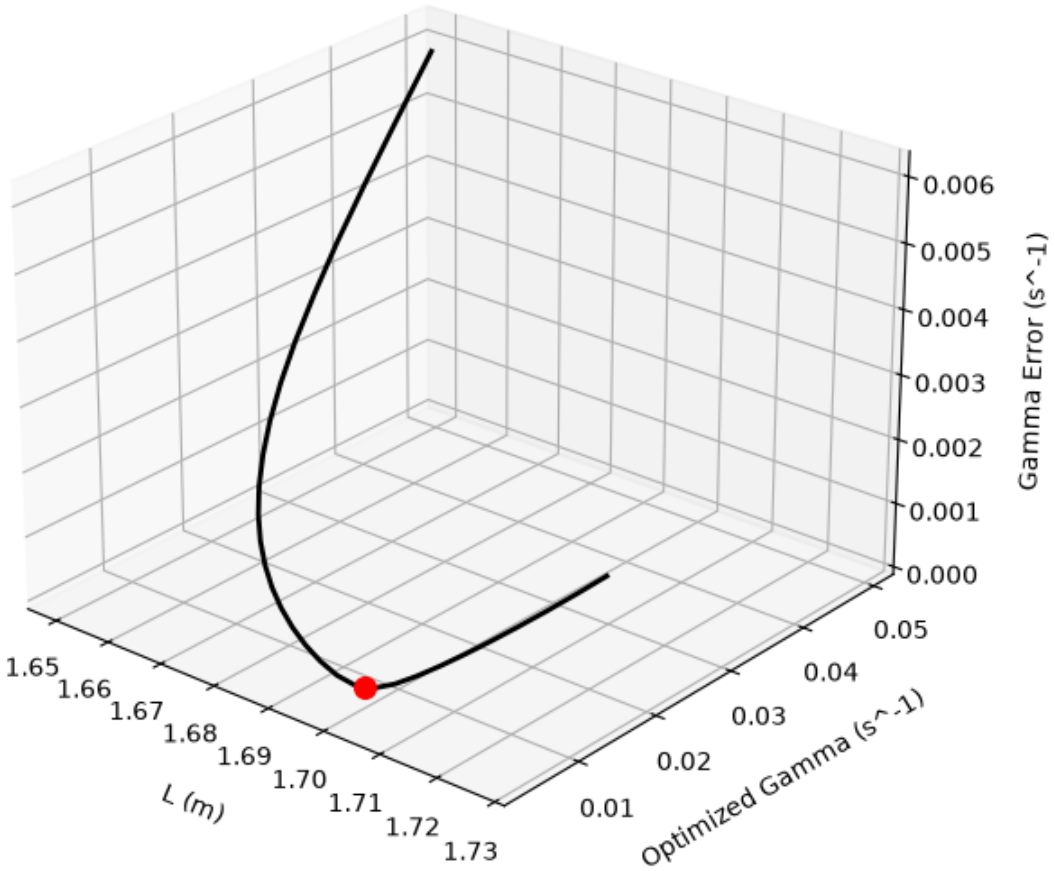
# plot 3D trajectory line
ax.plot(L_values, gamma_values, gamma_errors, color='black', linewidth=2)

# plot singular red dot at optimal 2-parameter solution
ax.scatter([L_fit], [gamma_fit], [gamma_err], color='red', s=70,␣
 ↪depthshade=False)

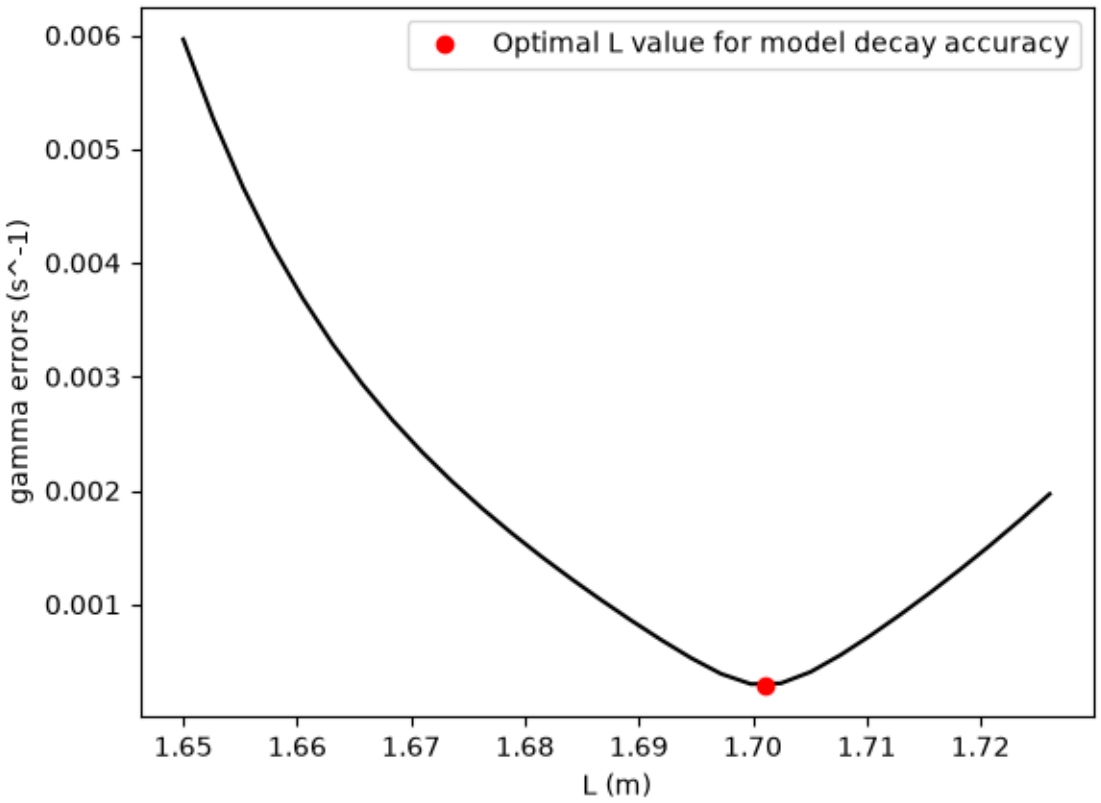
ax.set_xlabel('L (m)', labelpad=10)
ax.set_ylabel('Optimized Gamma (s^-1)', labelpad=10)
ax.set_zlabel('Gamma Error (s^-1)', labelpad=10)

# Fixed camera angle tuned to highlight the minimum
ax.view_init(elev=25, azim=-50)
```

```
plt.show()
```



[14]:
```
plt.plot(L_values, gamma_errors, color = 'black')
plt.plot(L_fit, gamma_err, 'ro', markersize = 6, label = 'Optimal L value for␣
 ↪model decay accuracy')
plt.xlabel('L (m)')
plt.ylabel('gamma errors (s^-1)')
plt.legend()
plt.show()
```



The curve is broad and has regions of strong sloped, meaning L and γ are coupled. The reported "best" value depends heavily on which value of the other parameter was assumed.

Another way to visualize the coupling, is to make a loss/error heatmap over (γ, L). A well-identified problem would have a compact, circular minimum. Parameter coupling appears as a long, narrow diagonal valley:

[15]:
```
plt.close('all') # close any open figures

time_data = subset_df["Time"].to_numpy()
angle_data = subset_df["Angle"].to_numpy()

# Parameter ranges to investigate
gamma_values = np.linspace(0.0, 0.05, 100)
L_values = np.linspace(1.64, 1.75, 100)

# Store sum-of-squared errors for every (gamma, L) pair
sse = np.empty((len(gamma_values), len(L_values)))

for i, gamma in enumerate(gamma_values):
    for j, L in enumerate(L_values):
        prediction = solve_pendulum_ivp(time_data, gamma, L)
```

```
        residuals = angle_data - prediction
        sse[i, j] = np.sum(residuals**2)

# Convert to a DataFrame for seaborn
error_surface = pd.DataFrame(
    sse,
    index=np.round(gamma_values, 3),
    columns=np.round(L_values, 3),
)

# Use log10 so the shallow error valley is easier to see
log_error_surface = np.log10(error_surface)

# Locate the best parameter combination
best_index = np.unravel_index(np.argmin(sse), sse.shape)
best_gamma = gamma_values[best_index[0]]
best_L = L_values[best_index[1]]
best_sse = sse[best_index]

print(f"Best gamma: {best_gamma:.5f}")
print(f"Best L:     {best_L:.5f}")
print(f"Minimum SSE: {best_sse:.6g}")

# Plot the error surface
plt.figure(figsize=(11, 7))

ax = sns.heatmap(
    log_error_surface,
    cmap="viridis",
    cbar_kws={"label": r"$\log_{10}(\mathrm{SSE})$"},
    xticklabels=5,
    yticklabels=5,
)

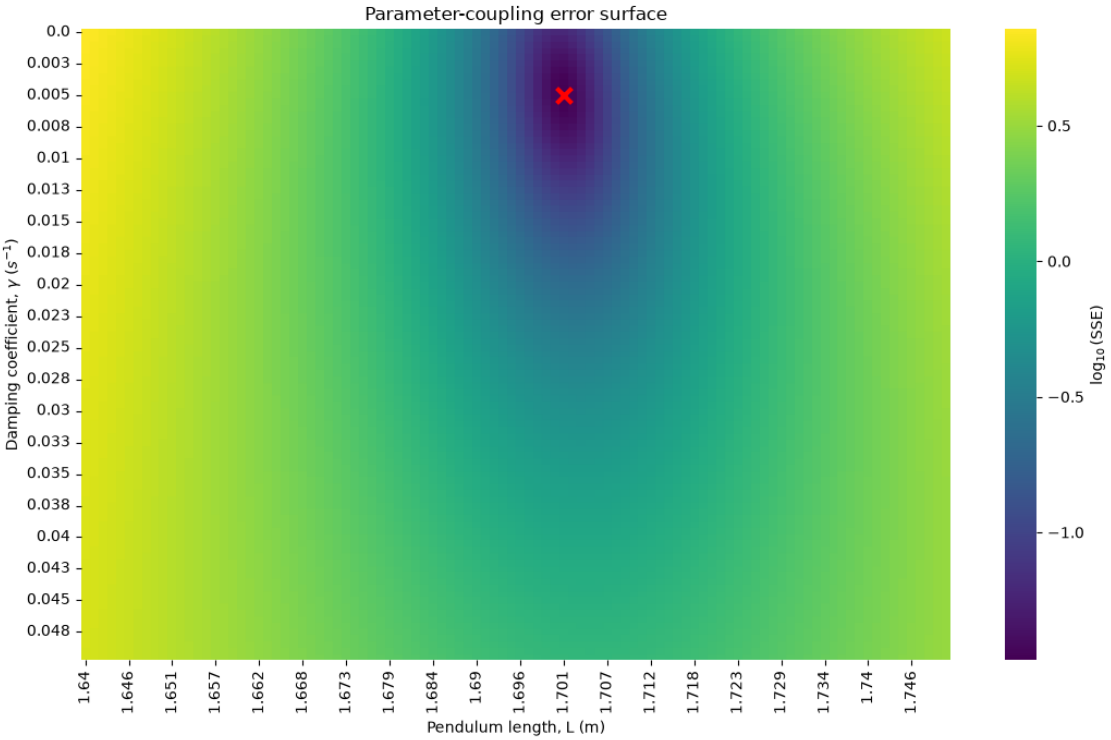
# Mark the best-fit location
best_row = best_index[0] + 0.5
best_col = best_index[1] + 0.5
ax.scatter(best_col, best_row, color="red", marker="x", s=100, linewidths=3)

ax.set_xlabel("Pendulum length, L (m)")
ax.set_ylabel(r"Damping coefficient, $\gamma$ ($s^{-1}$)")
ax.set_title("Parameter-coupling error surface")

plt.tight_layout()
plt.show()
```

```
Best gamma: 0.00505
```

```
Best L:     1.70111
Minimum SSE: 0.0337406
```



Increasing one parameter can be compensated for by changing the other. The optimizer may find one point in the valley, but many nearby points are almost equally plausible. FIM would summarize the local shape and orientation of that valley and identify the poorly constrained direction.

7 Explanation of major changes compared to proposal:

The data that we have chosen to work with is the pendulum data collected by Sergey during his undergraduate. It was initially hypothesised in the proposal that a more sparse data set would improve model accuracy, so the original concept idea was that FIM would be used to inform D-Optimal design. We have already shown that adding an additional free parameter improves model accuracy, so creating a subset with D-optimal design is no longer necessary. Our major scope change is that we are no longer focused on improving model accuracy, but rather on decoupling model parameters while preserving accuracy. FIM is still very much needed for our project, as it provides a sensitivity matrix to study parameter correlation; it is simply no longer used to aid D-optimal design.

8 References

Groeneveld, E. A reparameterization to improve numerical optimization in multivariate REML (co)variance component estimation. Genet Sel Evol 26, 537 (1994). https://doi.org/10.1186/1297-

9686-26-6-537

9 Statements of contributions of each team member (1-2 sentences):

Angela DiAndreth: Created the interactive slider plot to show changing of the parameters L and . Wrote for the explanation of major changes, statement of proposal match, and critical challenge sections and reformatted PoC into required assignment format.

Sergey Dutt: Provide dataset, created fix_data.py and wrote the code for the following functions: pendulum_model, solve_pendulum_ivp, and plot_model. Wrote Mathematical basis, and aided in the revisions of all other sections (heavy revisions on Critical Challenge).

Edward Lombo-Becerra: Adjusted graph formattings, wrote heatmap plot analysis code, wrote guided descriptions through Solution to reduced problem section

[ ]: